



TrafficAgenticAI light management with SUMO visual simulation.

Raffaele Neri - Matteo Melotti - Marco Crisafulli
26/05/2026



I sistemi semaforici **tradizionali** utilizzano tempi fissi e reagiscono al traffico solo dopo la formazione di congestioni.

Questo progetto introduce un **nuovo paradigma**: sfruttare un sistema **Agentic AI** per trasformare la gestione del traffico da sistema statico a sistema intelligente, adattivo e collaborativo.

Obiettivi del progetto

01

Ottimizzare il flusso del traffico urbano in tempo reale

Adattare dinamicamente i cicli semaforici in base alle condizioni del traffico rilevate dagli agenti.

02

Ridurre congestione e tempi di attesa ai semafori

03

Coordinare più incroci tramite agenti AI autonomi

04

Visualizzare e testare scenari di traffico con SUMO



Obiettivi del progetto

01 Ottimizzare il flusso del traffico urbano in tempo reale

02 **Ridurre congestione e tempi di attesa ai semafori**
Minimizzare code e rallentamenti migliorando la distribuzione dei veicoli nelle intersezioni.

03 Coordinare più incroci tramite agenti AI autonomi

04 Visualizzare e testare scenari di traffico con SUMO



Obiettivi del progetto

01 Ottimizzare il flusso del traffico urbano in tempo reale

02 Ridurre congestione e tempi di attesa ai semafori

03 **Coordinare più incroci tramite agenti AI autonomi**
Permettere agli agenti di comunicare e collaborare per gestire il traffico su larga scala.

04 Visualizzare e testare scenari di traffico con SUMO



Obiettivi del progetto

01 Ottimizzare il flusso del traffico urbano in tempo reale

02 Ridurre congestione e tempi di attesa ai semafori

03 Coordinare più incroci tramite agenti AI autonomi

04 **Visualizzare e testare scenari di traffico con SUMO**
Simulare ambienti urbani realistici per analizzare prestazioni e comportamento del sistema AI.



Contesto generale

Il sistema Agentic AI è stato testato in ambienti urbani simulati tramite **SUMO** (Simulation of Urban Mobility).

Il sistema è basato su **agenti AI autonomi** capaci di percepire l'ambiente, prendere decisioni e collaborare tra loro.

Approccio intelligente e distribuito per la **gestione dinamica del traffico** urbano in tempo reale.

Architettura multi-agente progettata per coordinare incroci e ottimizzare i flussi veicolari.

Integrazione tra Agentic AI e simulazione SUMO per analizzare scenari urbani realistici e adattivi.



Contesto generale

Il sistema Agentic AI è stato testato in ambienti urbani simulati tramite **SUMO** (Simulation of Urban Mobility).

La combinazione di simulazione e agenti intelligenti garantisce:

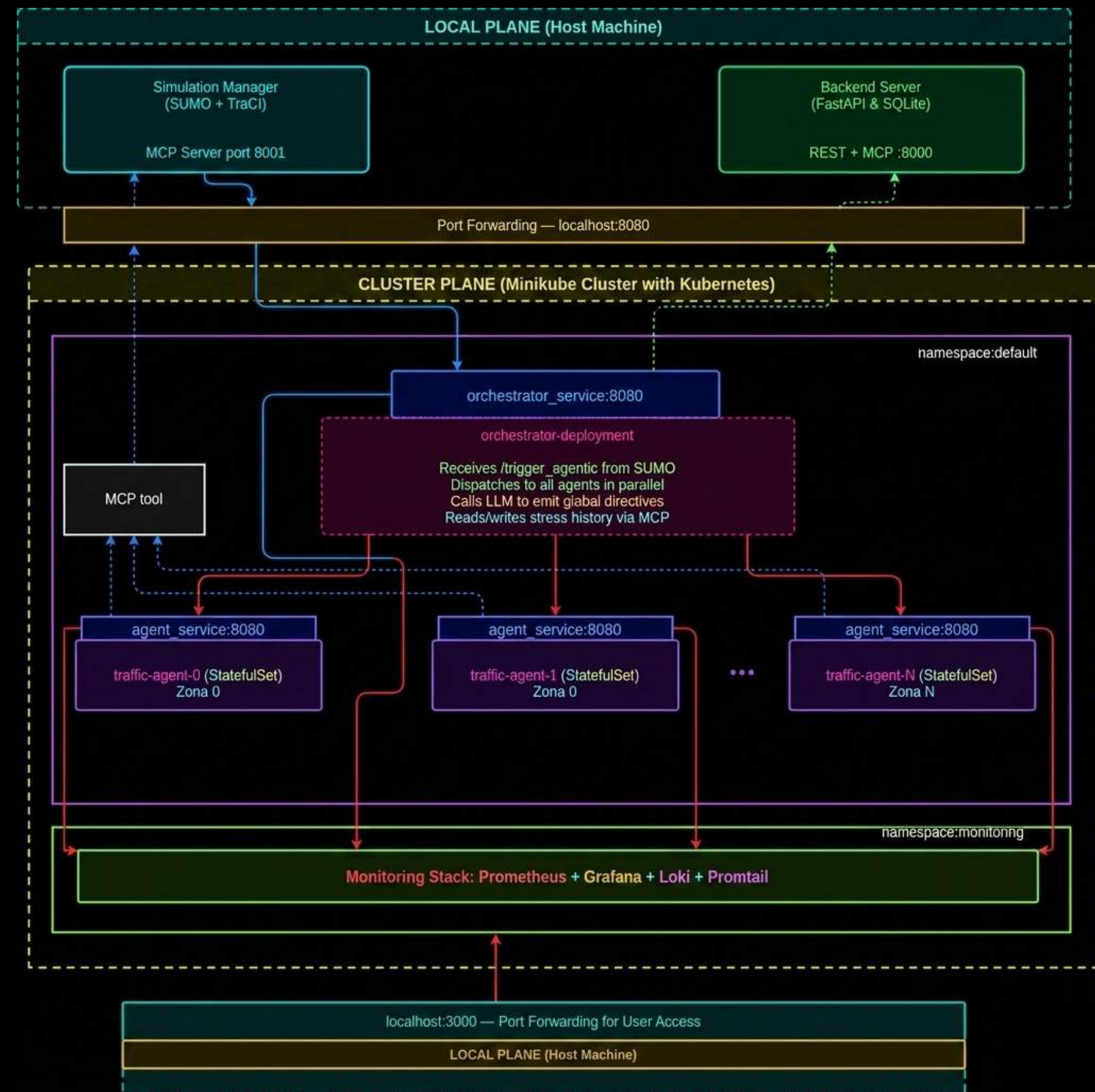
Adattabilità dinamica → gestione del traffico in tempo reale in base alle condizioni stradali.

Coordinazione distribuita → collaborazione tra più agenti per ottimizzare incroci e flussi urbani.

Analisi realistica → simulazione di scenari complessi e valutazione delle prestazioni del sistema.



Architettura del sistema



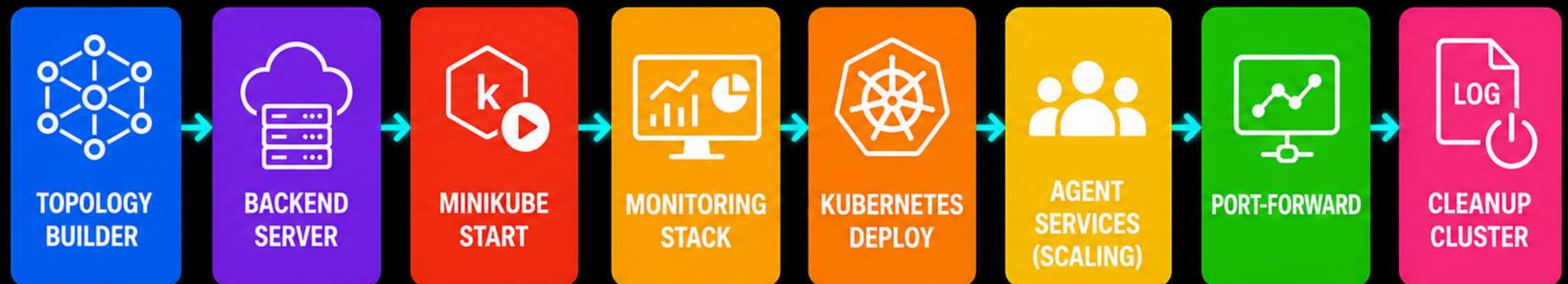
agenticTrafficManager.py

Il modulo **agenticTrafficManager.py** automatizza l'avvio dell'intera infrastruttura cluster.

Funzioni principali

- Genera le topologie degli agenti
- Avvia Minikube e Kubernetes
- Setup infrastruttura cluster
- Scala gli Agent Service in base a k
- Configura port-forward
- Gestisce cleanup finale del cluster

```
run_application(simulation_name, k, memory, cpus):  
  
    1. Genera le topologie  
    → build_topologies(simulation_name, k)  
  
    2. Carica le topologie nel backend  
    per ogni file JSON generato:  
    → ricava agent_id  
    → invia POST /api/topology/{agent_id}  
  
    3. Prepara l'infrastruttura  
    → avvia Minikube con memory e cpus  
    → installa monitoring stack  
    → costruisce immagini agent/orchestrator  
    → applica manifest Kubernetes  
    → scala traffic-agent a k repliche  
  
    4. Espone i servizi locali  
    → localhost:8080 → orchestrator-service  
    → localhost:3000 → Grafana  
  
    5. Mantiene il sistema attivo  
    finché l'utente non preme Ctrl+C  
  
    6. Cleanup finale  
    → chiude port-forward  
    → elimina risorse Kubernetes  
    → spegne Minikube
```



topology_builder.py & topology_library.py

Questi moduli si occupano di trasformare automaticamente la rete stradale di SUMO in topologie distribuite per gli agenti AI.

La pipeline legge il file **.net.xml**, estrae incroci, archi e semafori, applica un clustering geografico tramite K-Means e divide la città in aree indipendenti.

Per ogni area viene poi generata una topologia compatta in formato **JSON**, ottimizzata per **ridurre il consumo di token** durante il reasoning degli LLM.

Ogni TrafficAgent riceve quindi solo la porzione di rete necessaria per gestire il proprio traffico locale.



Minikube

Minikube viene utilizzato per eseguire localmente un cluster Kubernetes, permettendo di testare l'intera infrastruttura multi-agente senza dover usare un cluster cloud reale.

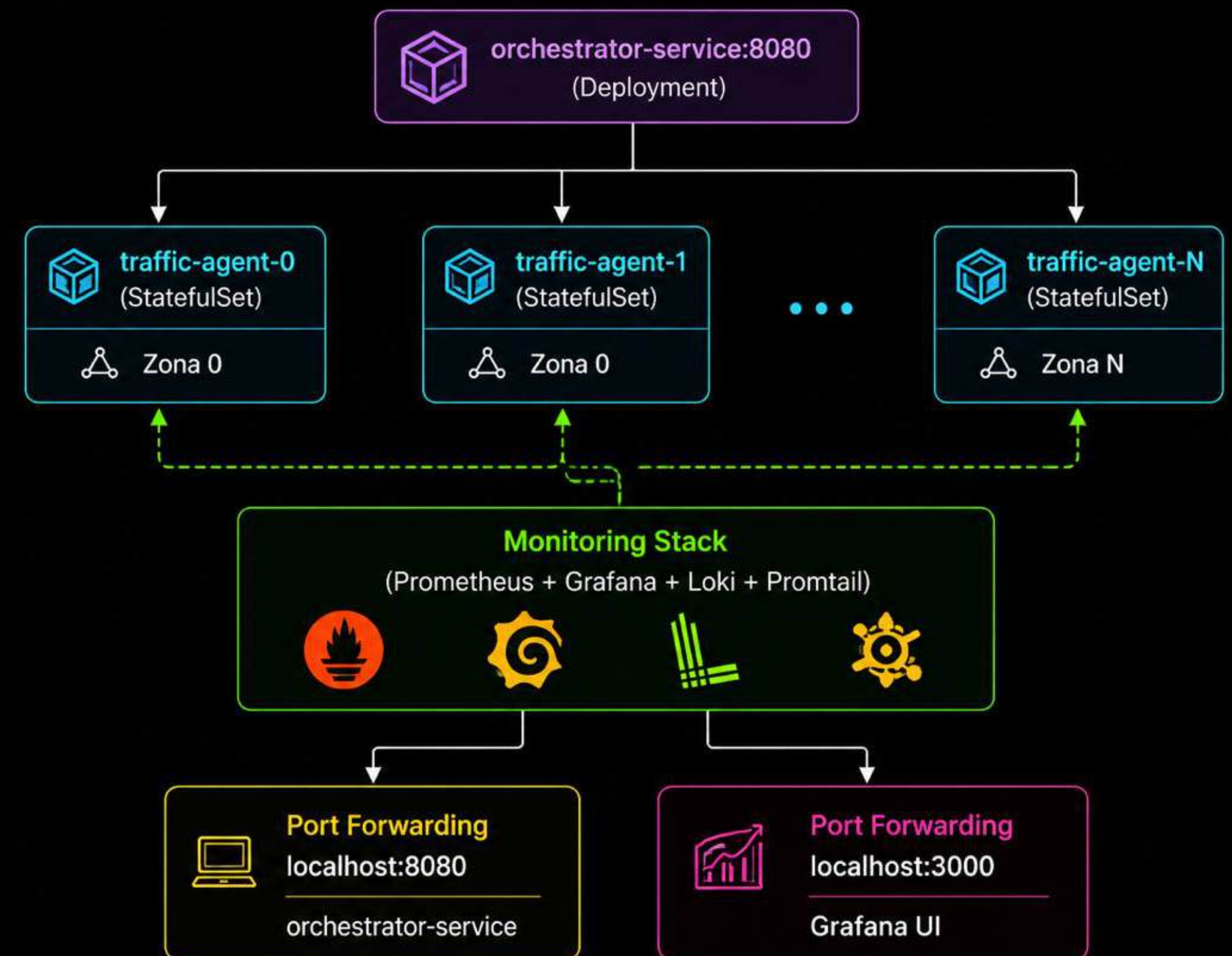
Minikube è stato scelto perché consente di sviluppare, testare e scalare il sistema in modo controllato e riproducibile.

Permette di:

- avviare Kubernetes in locale;
- testare Deployment, StatefulSet e Service;
- scalare gli agenti tramite il parametro k;
- verificare networking, port-forward e monitoring;
- ridurre complessità e costi rispetto a un cluster cloud.

Nel progetto, Minikube ospita i pod degli agenti, dell'orchestratore e dello stack di monitoraggio, simulando un ambiente Kubernetes completo sulla macchina locale.

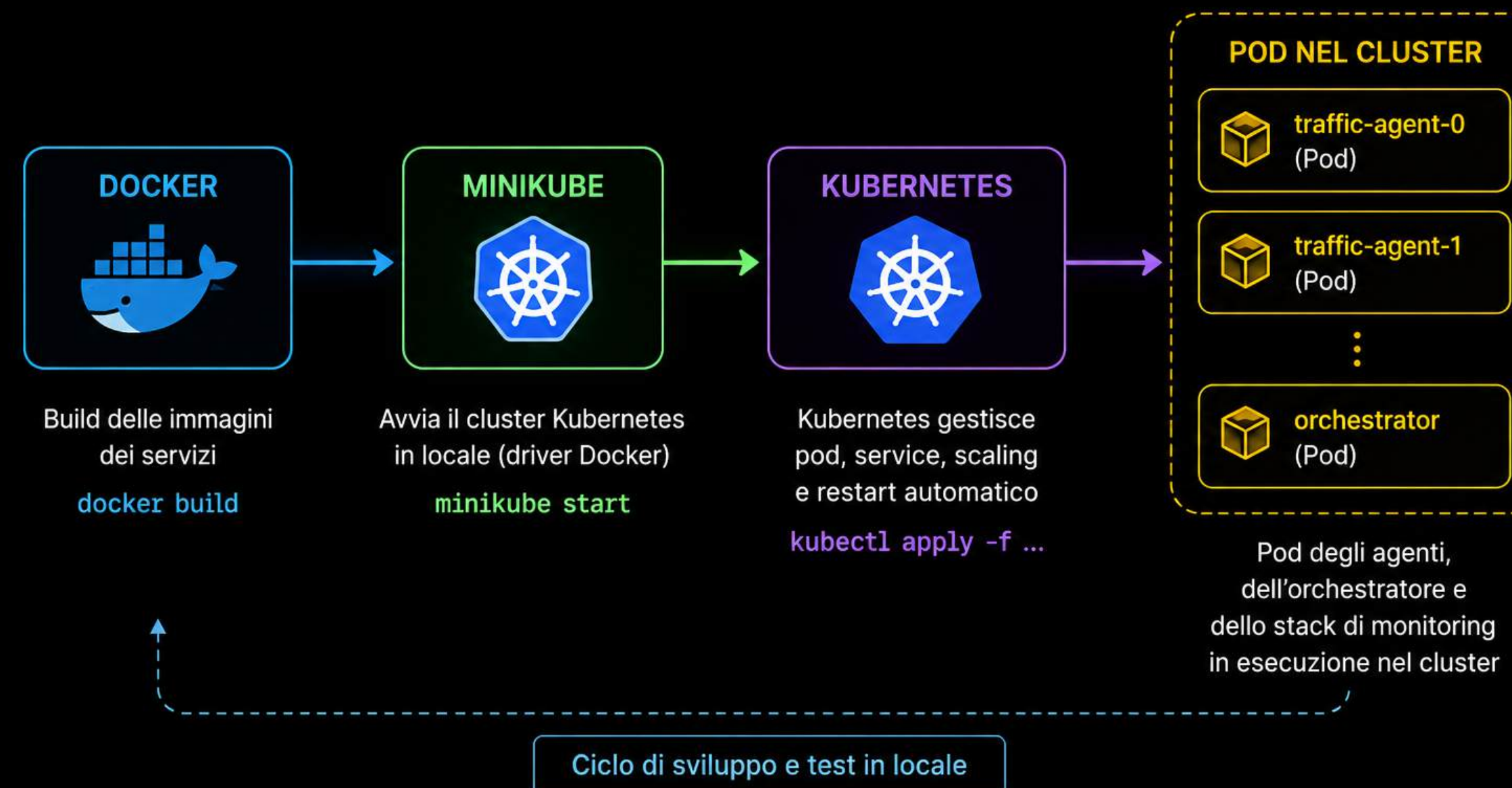
Nel progetto, Minikube ospita i pod degli agenti, dell'orchestratore e dello stack di monitoraggio, simulando un ambiente Kubernetes completo sulla macchina locale.



Minikube

Docker viene usato per costruire le immagini dei servizi, mentre Minikube crea il cluster Kubernetes locale usando Docker come driver.

Kubernetes, all'interno di Minikube, gestisce i pod, i service, lo scaling e il restart automatico dei componenti.



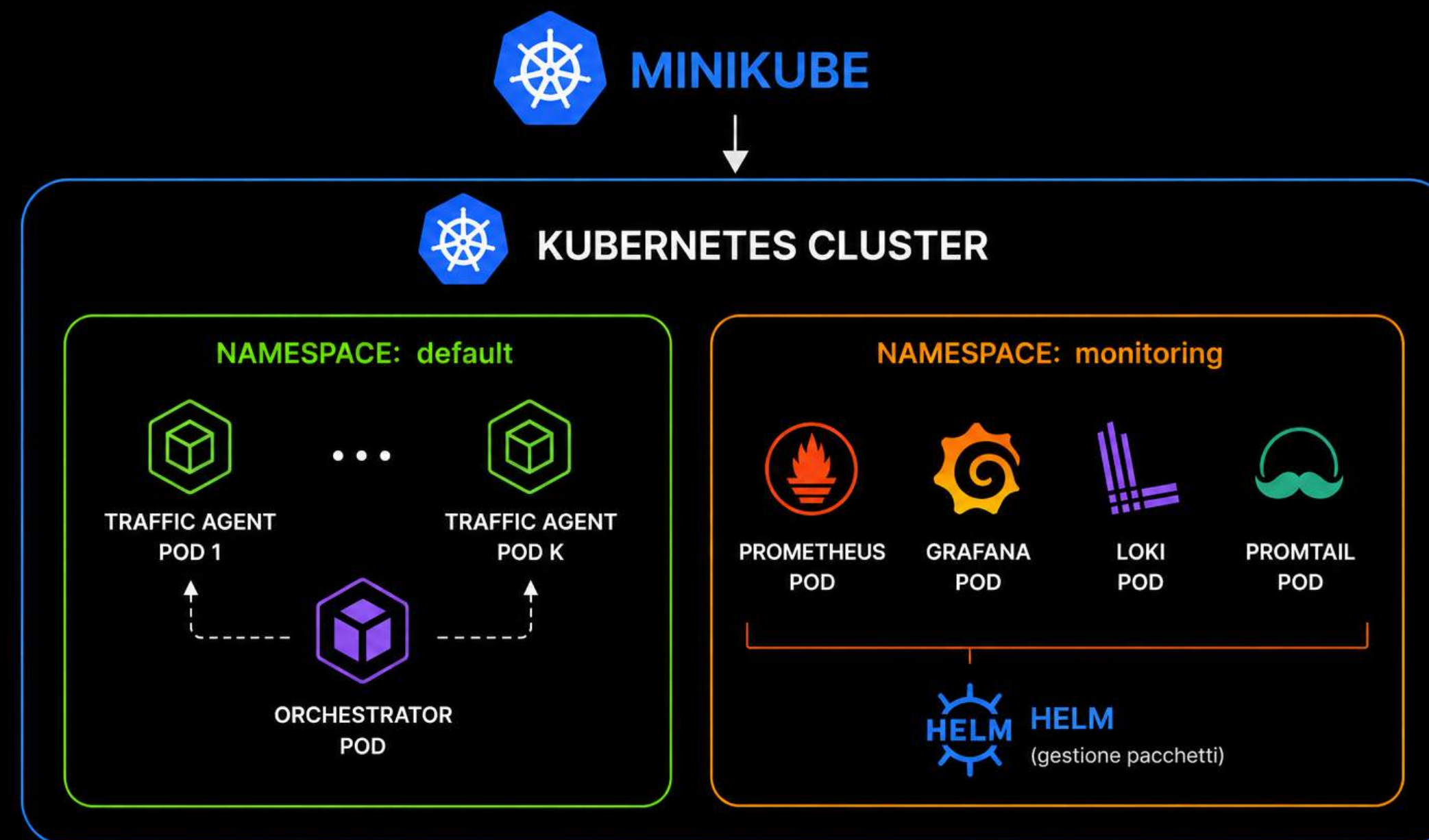
Architettura Kubernetes del Sistema

Una volta creato il cluster locale tramite Minikube, tutta l'infrastruttura viene gestita da Kubernetes.

Il sistema è suddiviso in due namespace principali:

default → contiene orchestrator e pod dei Traffic Agent

monitoring → contiene Prometheus, Grafana, Loki e Promtail installati tramite Helm.



Architettura Kubernetes del Sistema

Ogni Traffic Agent viene eseguito in un pod dedicato costruito a partire da immagini Docker generate dentro Minikube.

L'infrastruttura utilizza:

StatefulSet per i Traffic Agent, poiché ogni agente mantiene identità stabile e topologia associata;

Deployment per l'orchestratore, essendo un componente stateless.

Kubernetes garantisce:

- restart automatico dei pod in caso di fault;
- gestione delle risorse tramite Requests/Limits;
- discovery dinamica degli agenti tramite RBAC;
- caricamento sicuro delle API Key tramite Secret Kubernetes;
- connettività interna tramite policy DNS personalizzate.



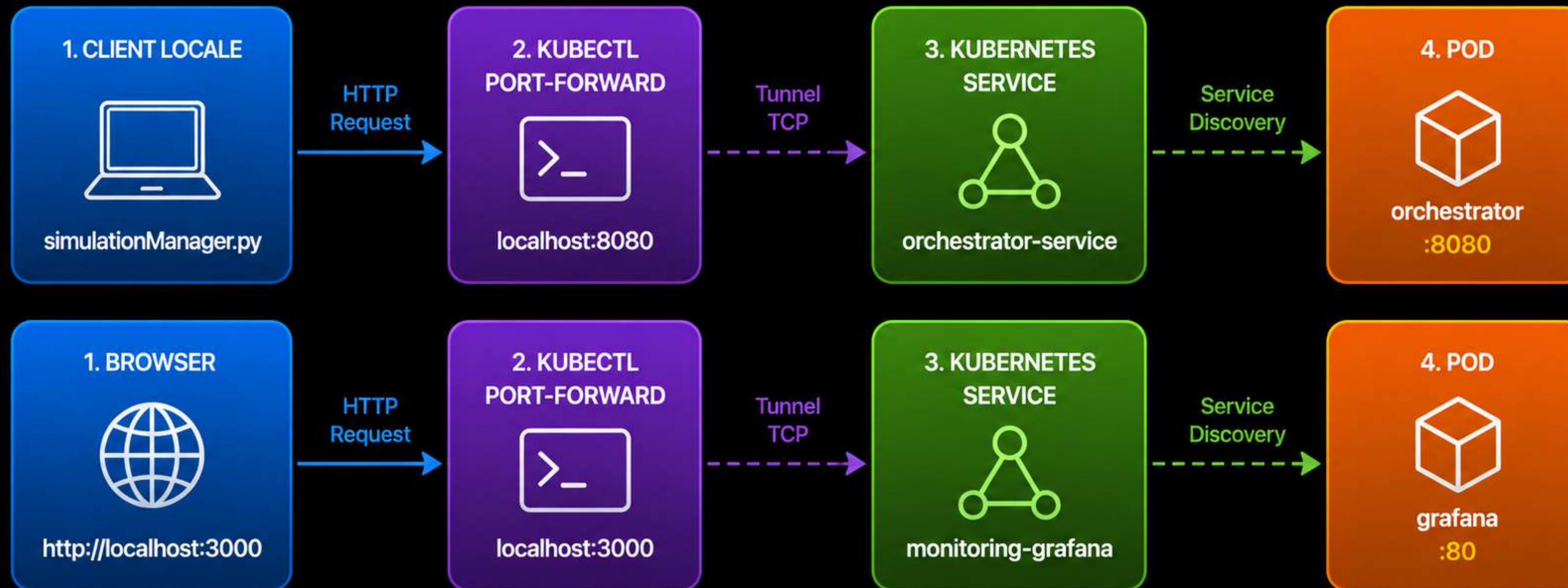
```
dnsPolicy: "None"
  dnsConfig:
    nameservers:
      - 10.96.0.10
      - 8.8.8.8
    searches:
      - default.svc.cluster.local
      - svc.cluster.local
      - cluster.local
```



```
apiVersion: rbac.authorization.k8s.io/v1
kind: Role
metadata:
  name: orchestrator-role
rules:
- apiGroups: [""]
  resources: ["pods"]
  verbs: ["get", "list", "watch"]
```



Port Forwarding



Il traffico dalla **macchina locale** viene instradato tramite **kubectl port-forward** al **Service** corrispondente nel cluster. Kubernetes si occupa di inoltrare la richiesta al **Pod** corretto tramite il meccanismo di **Service Discovery**.



Simulation Manager

- Il modulo **simulationManager.py** rappresenta il punto di ingresso della simulazione urbana e coordina l'interazione tra SUMO e il sistema multi-agente.
- Si occupa dell'avvio della simulazione, dell'aggiornamento continuo dello stato del traffico e dell'**esecuzione dei comandi provenienti dal sistema AI tramite MCP Server**.
- Durante ogni step vengono raccolti dati su veicoli, code e corsie controllate dai semafori, aggiornando una memoria condivisa utilizzata dagli agenti per il reasoning locale.
- Dopo un intervallo configurabile, il Simulation Manager contatta l'Orchestrator Service e avvia il loop agentico distribuito che continuerà automaticamente per tutta la durata della simulazione.

```
response = requests.post(
    ORCHESTRATOR_URL,
    json={
        "step": 0,
        "simulation_id": simulation_name
    }
)
```



Backend Server

Il modulo **backend_server.py** gestisce la persistenza dei dati del sistema tramite FastAPI, SQLite e MCP.

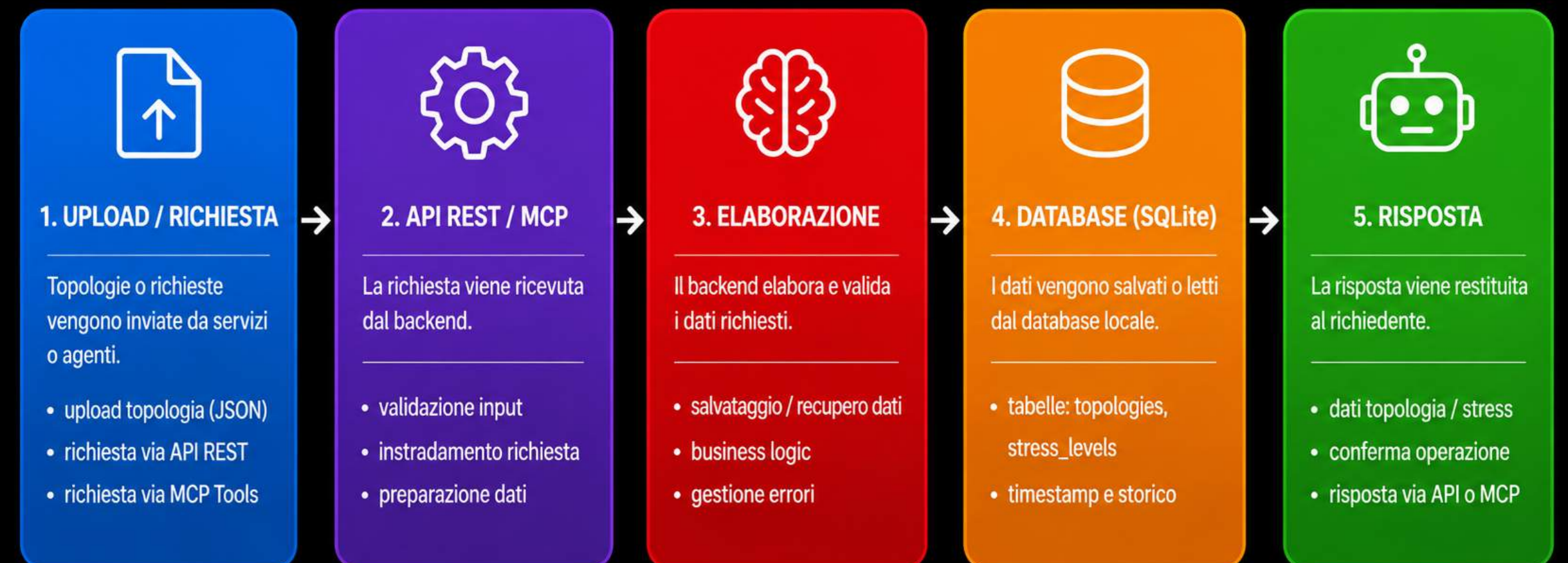
Ruolo principale

Memorizza topologie degli agenti e storico dello stress, rendendoli disponibili sia tramite API REST sia tramite tool MCP.

Funzioni principali

- Upload e recupero topologie JSON
- Salvataggio dati in database SQLite
- Persistenza dello storico stress degli agenti
- Esposizione API REST per i servizi
- Esposizione MCP tool per orchestratore

```
cursor.execute(
    "INSERT INTO stress_levels "
    "(agent_id, stress_value, prompt_text, timestamp) "
    "VALUES (?, ?, ?, ?)",
    (agent_id, stress_index, prompt_text, datetime.now())
)
```



Backend Server

Il modulo **backend_server.py** gestisce la persistenza dei dati del sistema tramite FastAPI, SQLite e MCP.

Ruolo principale

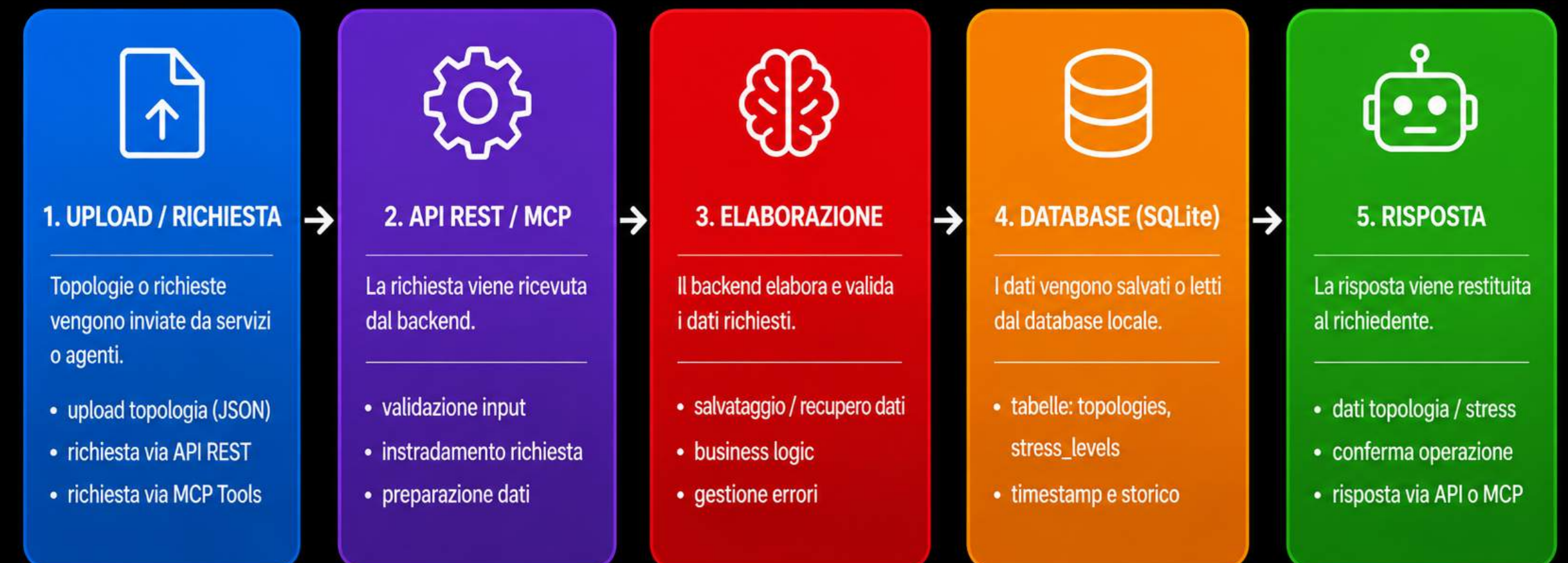
Memorizza topologie degli agenti e storico dello stress, rendendoli disponibili sia tramite API REST sia tramite tool MCP.

```
@app.get("/api/topology/{agent_id}")
async def get_topology(agent_id: str):
    ...

@mcp.tool()
def get_recent_stress(limit: int):
    ...
```

Endpoint / Tool principali

- POST /api/topology/{agent_id}
- GET /api/topology/{agent_id}
- write_topology()
- save_agent_stress()
- get_recent_stress()

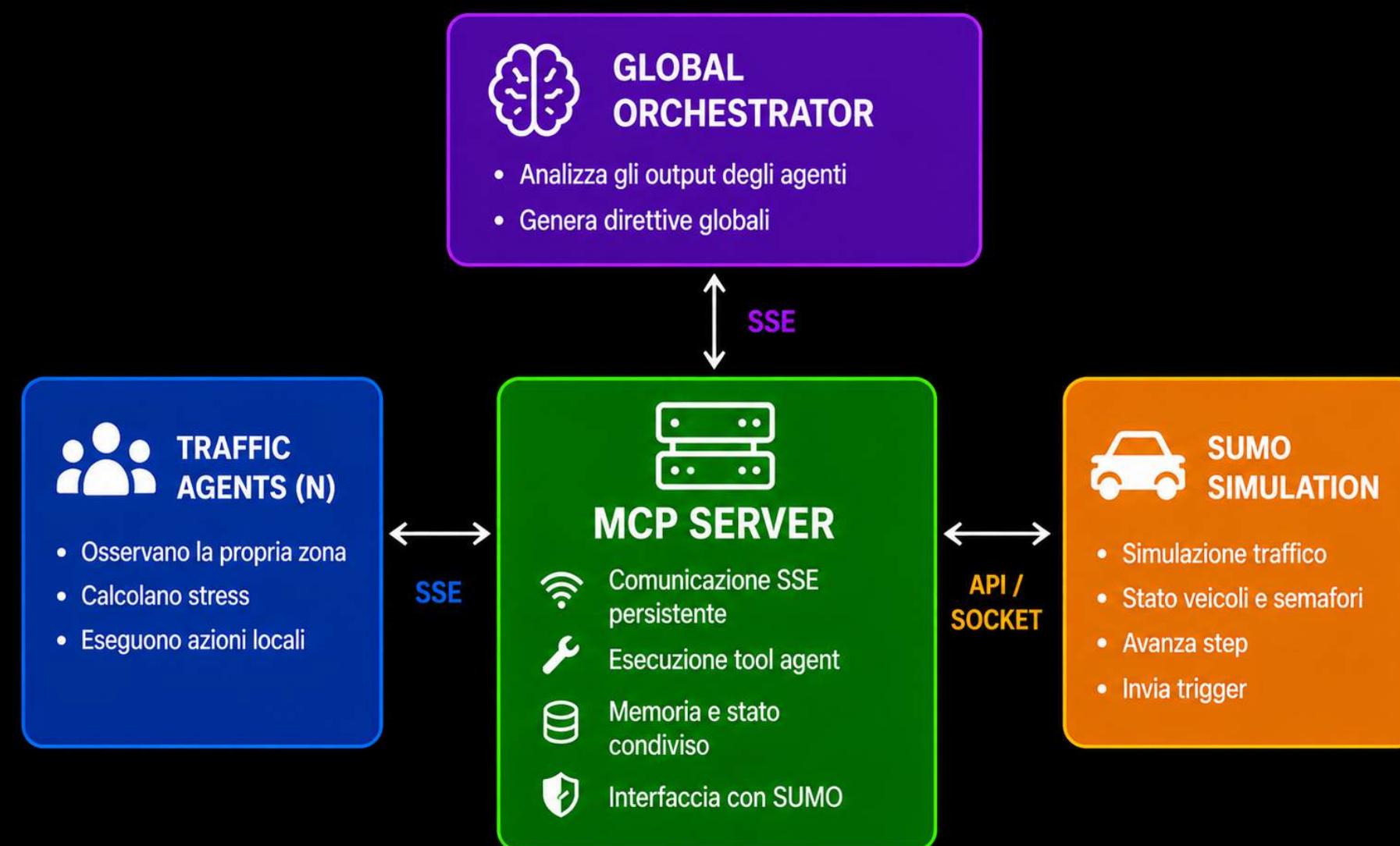


MCP e comunicazione

Il modulo MCP rappresenta il livello di comunicazione tra orchestratore, TrafficAgent e simulazione SUMO. Permette agli agenti AI di eseguire tool, recuperare dati e applicare modifiche ai semafori in tempo reale.

Il modulo MCP gestisce la comunicazione persistente tra orchestratore, TrafficAgent e simulazione SUMO tramite SSE.

Si occupa dell'esecuzione dei tool richiesti dagli agenti AI, fungendo da **interfaccia** tra il sistema intelligente e la simulazione del traffico. Inoltre centralizza l'accesso condiviso a memoria, stato dei semafori e metriche del traffico urbano.

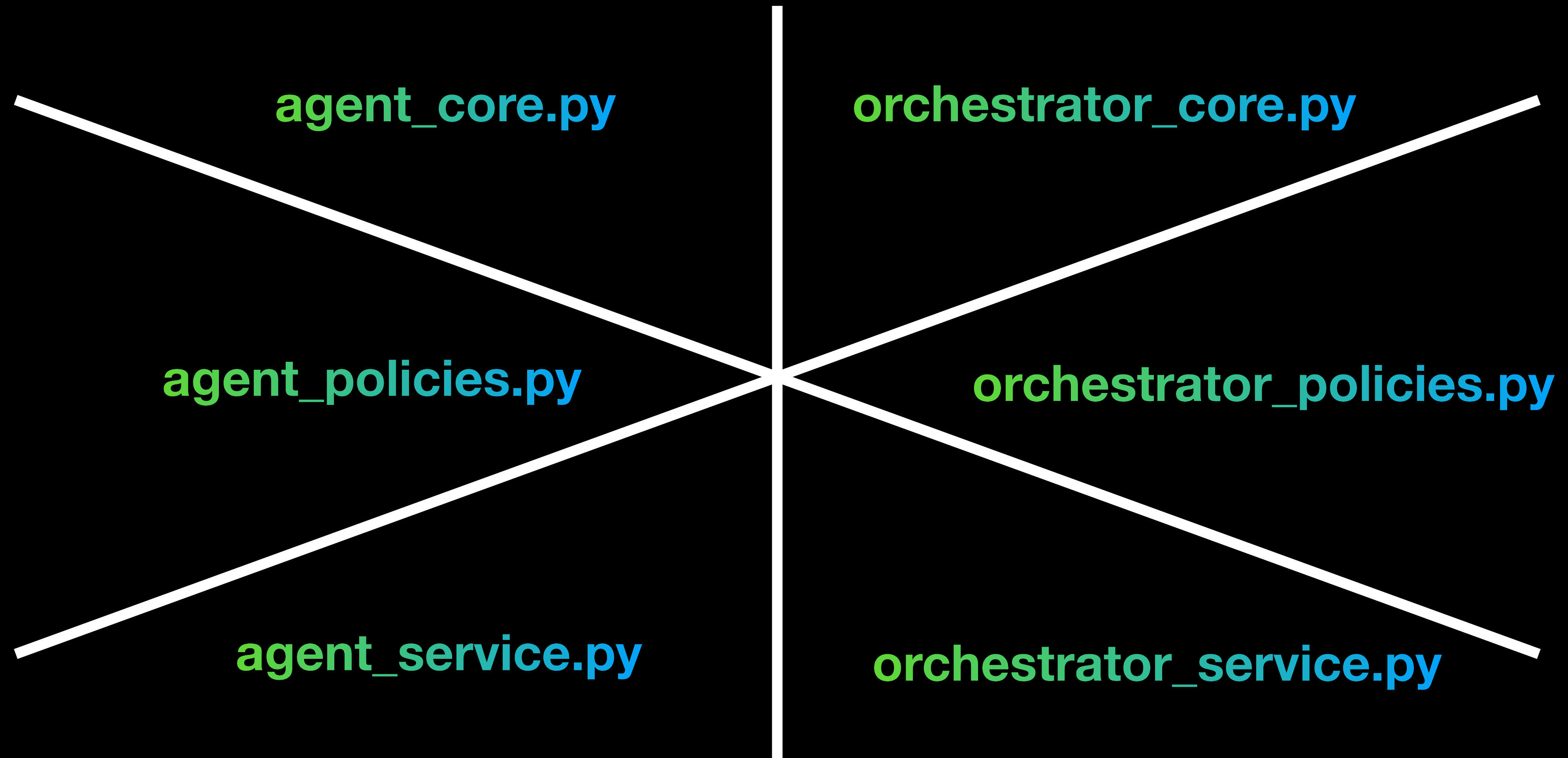


```
@mcp.tool()
async def compute_stress_index(tls_ids: list[str]) -> float:
    response = await client.post(
        f"{SUMO_API_URL}/compute_stress_index",
        json={"tls_ids": tls_ids}
    )
    return float(response.json().get("stress_index", 0.0))

@mcp.tool()
async def set_traffic_light_duration(tl_id: str, duration: float) -> dict:
    response = await client.post(
        f"{SUMO_API_URL}/set_traffic_light_duration",
        json={"tl_id": tl_id, "duration": duration}
    )
    return {"status": "ok", "tl_id": tl_id, "duration": duration}
```



Core Modules & Decision Policies



agent_core.py + agent_policies.py + agent_service.py

Questi moduli gestiscono il comportamento locale degli agenti AI associati agli incroci.

Il modulo **agent_core.py** rappresenta il nucleo operativo dei TrafficAgent.

Ogni agente controlla uno o più incroci e prende decisioni autonome basate sullo stato del traffico simulato.

Il modulo si occupa di :

- Gestione degli incroci assegnati all'agente
- Connessione persistente con MCP Server tramite SSE
- Esecuzione del **loop decisionale agentico**
- Invocazione di tool per analisi e controllo semaforico



agent_core.py + agent_policies.py + agent_service.py

Questi moduli gestiscono il comportamento locale degli agenti AI associati agli incroci.

Il modulo **agent_core.py** rappresenta il nucleo operativo dei TrafficAgent.

Ogni agente controlla uno o più incroci e prende decisioni autonome basate sullo stato del traffico simulato.

Tools MCP utilizzati

- `compute_stress_index()`
- `compute_phase_duration()`
- `set_traffic_light_duration()`
- `set_traffic_light()`



agent_core.py + agent_policies.py + agent_service.py

Questi moduli gestiscono il comportamento locale degli agenti AI associati agli incroci.

Il modulo agent_service.py espone ogni TrafficAgent come microservizio FastAPI indipendente.

Riceve le richieste dall'Orchestrator Service, avvia il reasoning locale in background e invia il risultato finale all'orchestratore.

Funzioni principali

- Recupera la topologia locale dal backend
- Inizializza il TrafficAgent
- Mantiene la connessione con il MCP Server
- Espone l'endpoint /evaluate
- Invia stress index, reasoning e azioni a /stress_comunication

```
@app.post("/evaluate")
async def evaluate_step(payload, background_tasks):
    background_tasks.add_task(
        app.state.agent_server.evaluate_and_push,
        step=payload.step,
        global_directive=payload.global_directive
    )

    return {"status": "processing"}
```



agent_core.py + agent_policies.py + agent_service.py

Questi moduli gestiscono il comportamento locale degli agenti AI associati agli incroci.

Il modulo agent_policies.py definisce il comportamento decisionale dei TrafficAgent tramite una system instruction fornita all'LLM.

Il prompt specifica:

- identità dell'agente e incroci gestiti
- direttiva globale ricevuta dall'Orchestrator
- tool MCP disponibili
- soglie di stress del traffico
- strategie operative per stress basso, moderato e alto
- regole anti-oscillazione per evitare cambi fase troppo frequenti

Output strutturato

L'LLM deve restituire sempre un JSON valido contenente:

- stress index calcolato
- reasoning tattico sintetico
- azioni applicate o suggerite sui semafori

```
FINAL OUTPUT RULES:
After evaluating all intersections and calling tools ONLY where necessary, output valid JSON only.

Reply ONLY with valid JSON.
No markdown.
No explanations outside JSON.

Exact format:
{{
  "stress_index": 0.0,
  "prompt_text": "short tactical reasoning",
  "actions_taken": [
    {{
      "action": "do_nothing|set_traffic_light_duration|set_traffic_light",
      "intersection_id": "intersection_id",
      "policy": "policy name or null",
      "phase_index_applied": 0,
      "duration_applied": 0,
      "reasoning": "short reason"
    }}
  ]
}}
```



orchestrator_core.py + orchestrator_policies.py + orchestrator_service.py

Questi moduli gestiscono il comportamento dell'orchestratore.

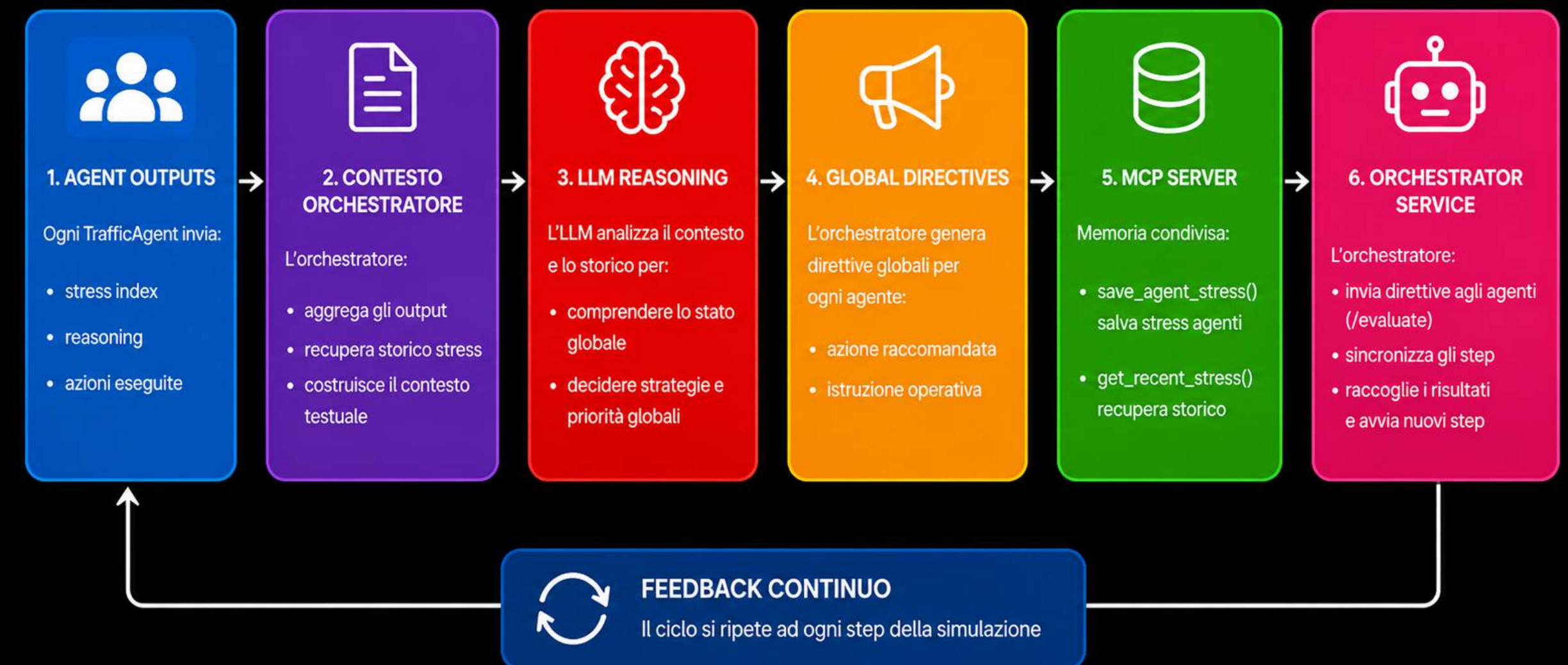
Il modulo **orchestrator_core.py** rappresenta il coordinatore globale del sistema multi-agente.

Funzioni principali

- Riceve gli output dei TrafficAgent
- Converte stress, azioni e reasoning in un contesto unico
- Salva e recupera lo storico tramite MCP
- Invia il contesto all'LLM
- Genera direttive globali per gli agenti
- Gestisce fallback in caso di errori o JSON non valido

Tool MCP usati

- `save_agent_stress()`
- `get_recent_stress()`



orchestrator_core.py + orchestrator_policies.py + orchestrator_service.py

Questi moduli gestiscono il comportamento dell'orchestratore.

Il modulo orchestrator_service.py gestisce il ciclo multi-agente e sincronizza l'avanzamento logico degli step.

Funzioni principali

- Riceve il trigger iniziale da SUMO
- Rileva automaticamente gli agenti attivi tramite Kubernetes
- Invia a ogni Agent Service lo step corrente e la direttiva globale
- Riceve stress index, reasoning e azioni locali dagli agenti
- Aggrega i risultati dello step
- Avvia il reasoning globale dell'orchestratore
- Salva la nuova direttiva globale
- Innesca automaticamente lo step successivo

Sincronizzazione

orchestrator_service.py implementa una barriera di sincronizzazione: il reasoning globale parte solo quando tutti gli agenti attivi hanno inviato il proprio risultato.

In questo modo lo step successivo viene avviato solo dopo aver raccolto una visione completa dello stato locale della rete.

```
is_complete, data = await app.state.orch.process_agent_data(
    payload.step,
    payload.agent_id,
    payload.result
)

if is_complete:
    next_step = await app.state.orch.workflow(
        payload.step,
        data
    )
```



orchestrator_core.py + orchestrator_policies.py + orchestrator_service.py

Questi moduli gestiscono il comportamento dell'orchestratore.

Orchestrator Policies

Il prompt definisce le regole strategiche dell'orchestratore globale.

L'orchestratore non modifica direttamente i semafori, ma invia direttive agli agenti locali.

Il prompt gestisce:

- livelli di stress del traffico
- storico recente degli agenti
- trend: miglioramento, stabilità o peggioramento
- priorità tra agenti con stress diverso
- prevenzione di direttive contraddittorie

Direttive possibili

- **prioritize_flow** → dare priorità al flusso
- **hold_or_balance** → mantenere equilibrio
- **reduce_aggressiveness** → ridurre interventi aggressivi

Output

Restituisce un JSON con reasoning globale e una direttiva per ogni agente.

```
{
  "global_reasoning": "short reasoning including current stress and trend",
  "directives": [
    {
      "target_agent": "agent id",
      "action": "prioritize_flow|hold_or_balance|reduce_aggressiveness",
      "instruction": "short instruction"
    }
  ]
}
```



Agenti custom vs Frameworks

✓ VANTAGGI

-  Pieno controllo sull'architettura
-  Maggiore ottimizzazione dei token
-  Integrazione diretta con MCP e logica di dominio
-  Comportamento prevedibile
-  Minore overhead
-  Migliore controllo su safety e guardrails
-  Più semplice integrare politiche personalizzate

✗ SVANTAGGI

-  Maggiore complessità implementativa
-  Gestione manuale di memoria e orchestration
-  Debugging più difficile
-  Meno componenti già pronti
-  Sviluppo più lento
-  Manutenzione maggiore

✓ VANTAGGI

-  Sviluppo rapido
-  Orchestration già pronta
-  Memory e tool management integrati
-  Multi-agent semplificato
-  Integrazione immediata con LLM provider
-  Ecosistema ampio

✗ SVANTAGGI

-  Overhead elevato
-  Minore controllo interno
-  Maggiore consumo token
-  Debugging più complesso
-  Astrazione eccessiva
-  Rischio di loop/hallucination meno controllabili
-  Difficile ottimizzazione fine-grained per sistemi realtime



llm_connector.py + adapter_connector.py

I moduli adapter_connector.py e llm_connector.py forniscono un'interfaccia unificata verso gli LLM, astrando le differenze tra provider e SDK, essi svolgono lo stesso compito per gli agenti e per l'orchestratore.

Obiettivi principali

- **Supporto multi-provider** → OpenAI, OpenRouter, LiteLLM e LM Studio locale
- **Standardizzazione delle risposte** → tramite UnifiedResponse e UnifiedFunctionCall
- **Gestione del tool calling loop** → per agenti e orchestratore

```
class UnifiedFunctionCall:
    def __init__(self, name, args, call_id=None):
        self.name = name
        self.args = args
        self.id = call_id

class UnifiedResponse:
    def __init__(self, text="", function_calls=None):
        self.text = text
        self.function_calls = function_calls or []

class OpenAIChatWrapper:
    async def send_message(self, message):
        response = await self.client.chat.completions.create(
            model=self.model,
            messages=self.history,
            tools=self.tools
        )

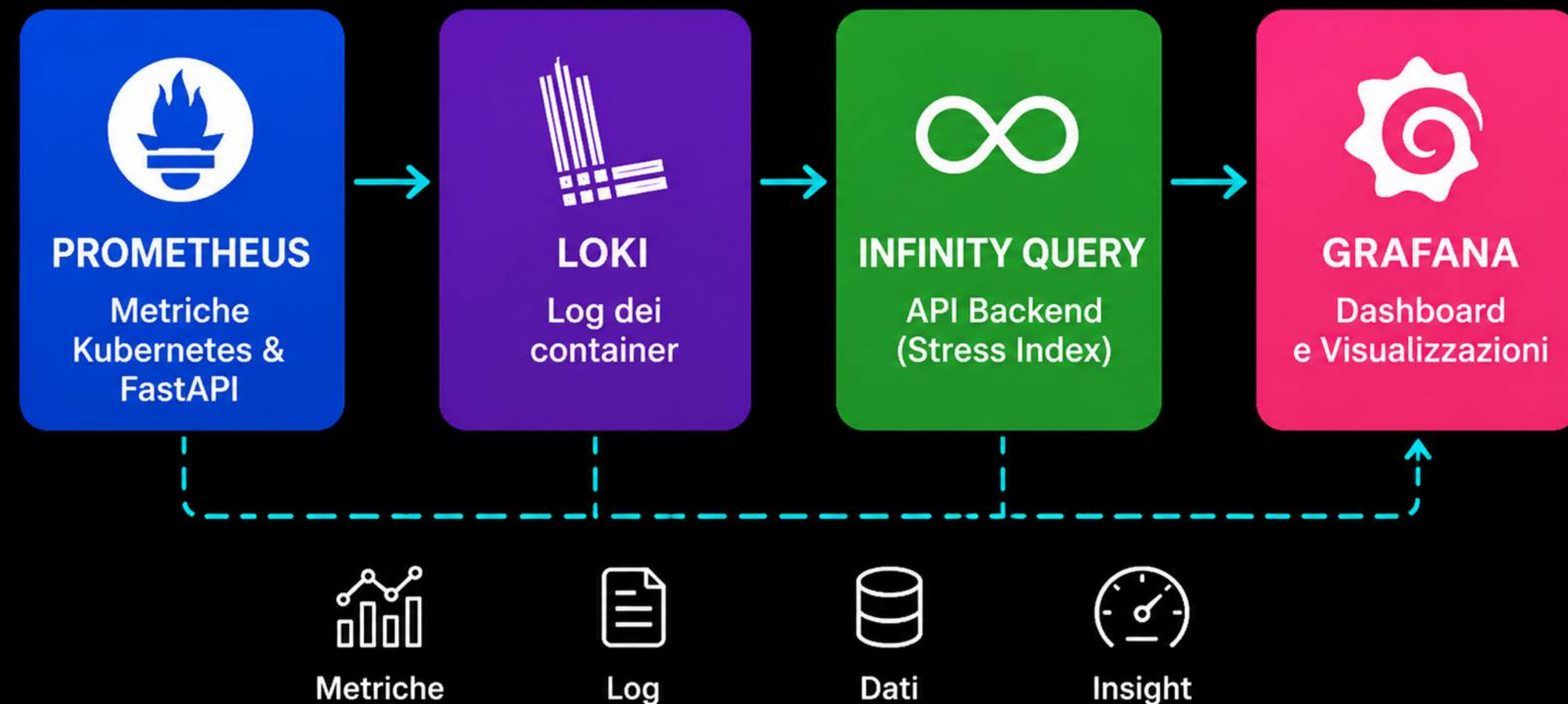
        return UnifiedResponse(
            text=response.choices[0].message.content
        )

class AgentConnector:
    def create_agentic_chat(self, system_instruction, openai_tools):
        return OpenAIChatWrapper(
            self.client,
            self.model,
            system_instruction,
            openai_tools
        )
```

Monitoring con Grafana

Grafana è il punto centrale di osservabilità del sistema.

Lo stack viene installato automaticamente tramite Helm e integra Prometheus, Loki, Promtail e Infinity Query per monitorare cluster, servizi e metriche applicative.

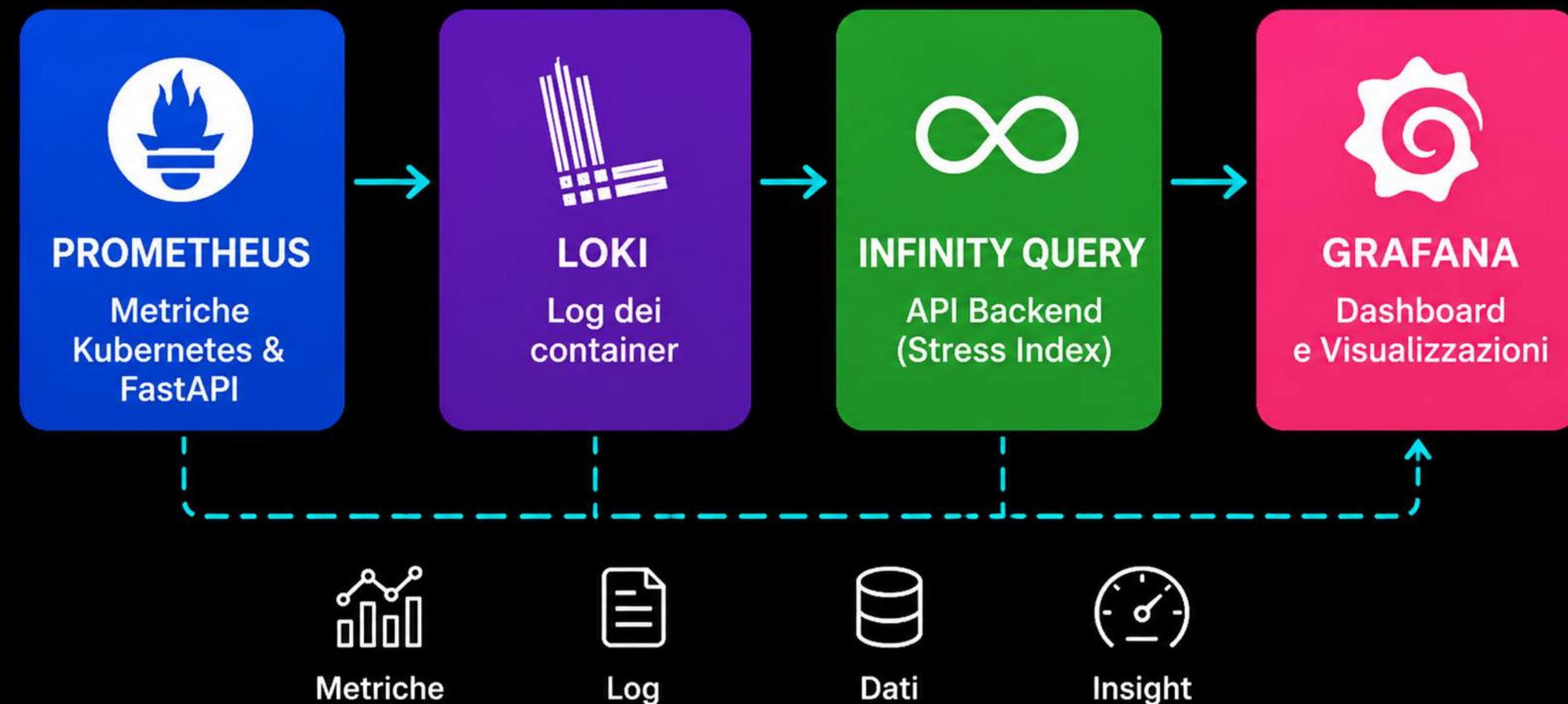


Monitoring con Grafana

Prometheus raccoglie metriche numeriche da Kubernetes e dagli endpoint FastAPI.

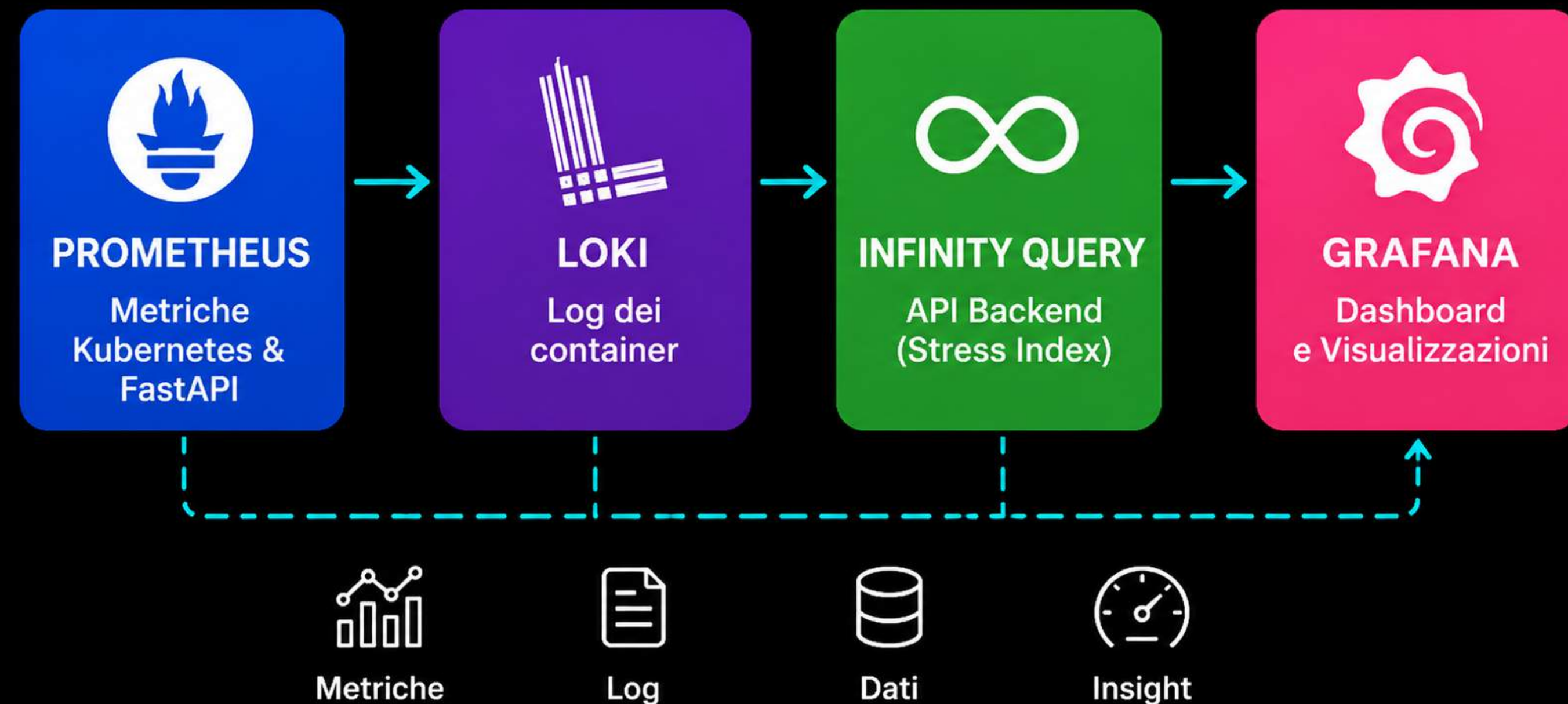
Loki, tramite Promtail, raccoglie i log dei container.

Infinity Query interroga il backend REST per recuperare lo storico degli `stress_index` salvati in SQLite.



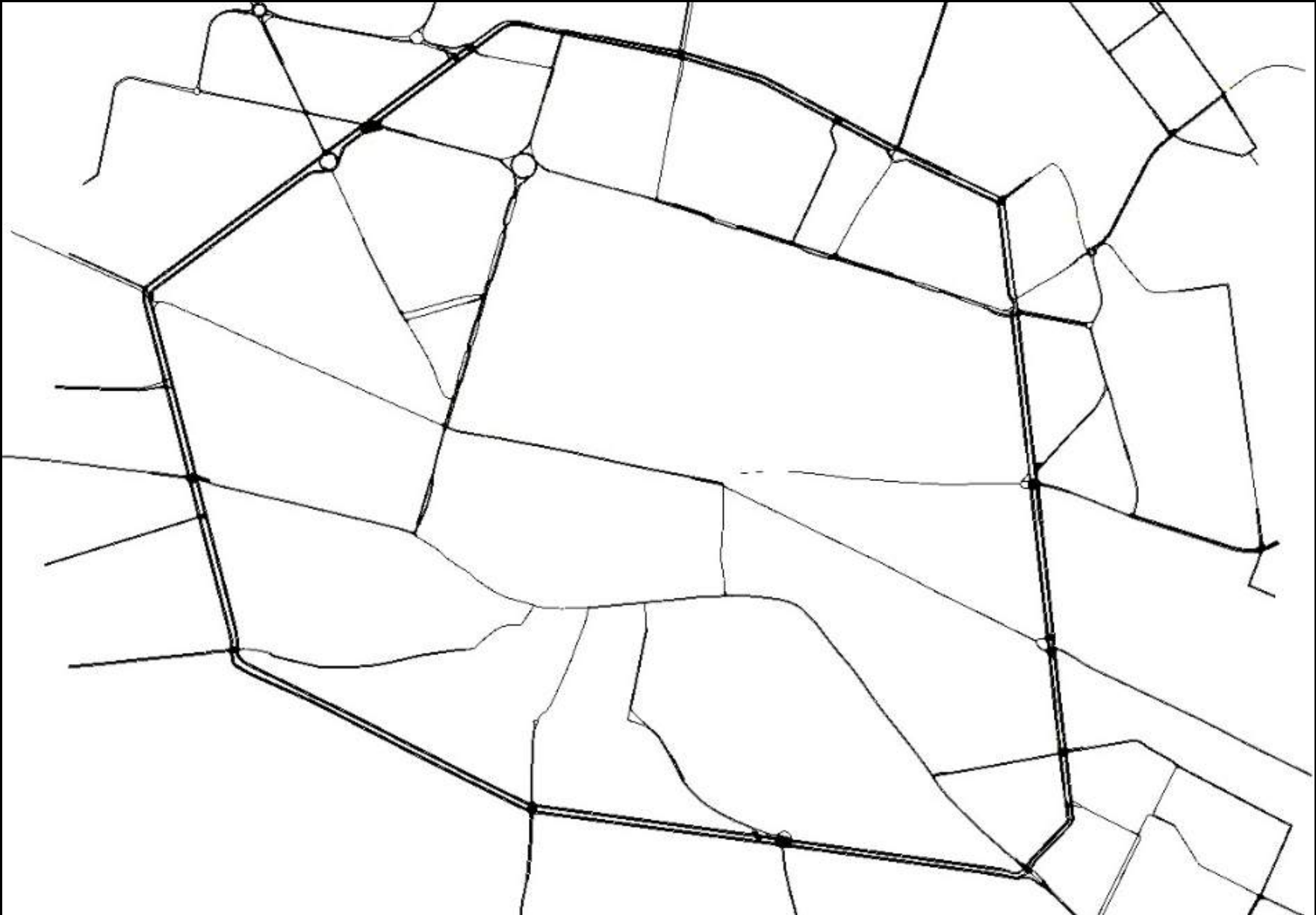
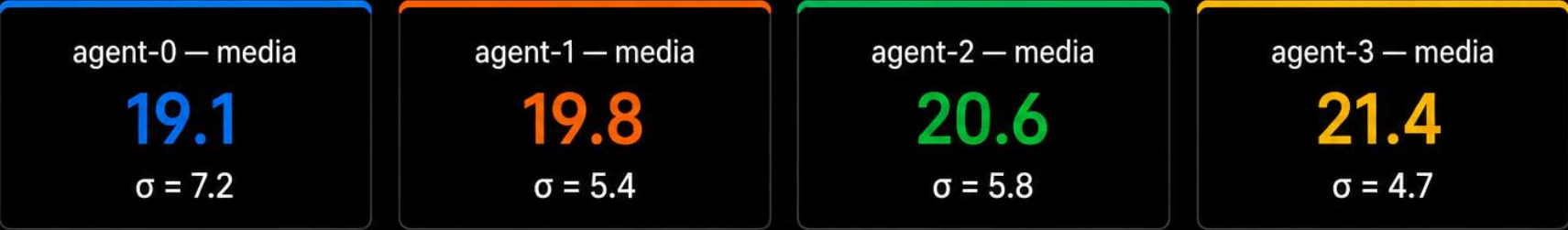
Monitoring con Grafana

Le dashboard mostrano pod attivi, richieste HTTP, tempi di risposta, log applicativi e andamento dello stress_index per agente. Questo permette di individuare anomalie, errori di comunicazione, congestioni improvvise e valutare l'efficacia delle decisioni dell'orchestratore.



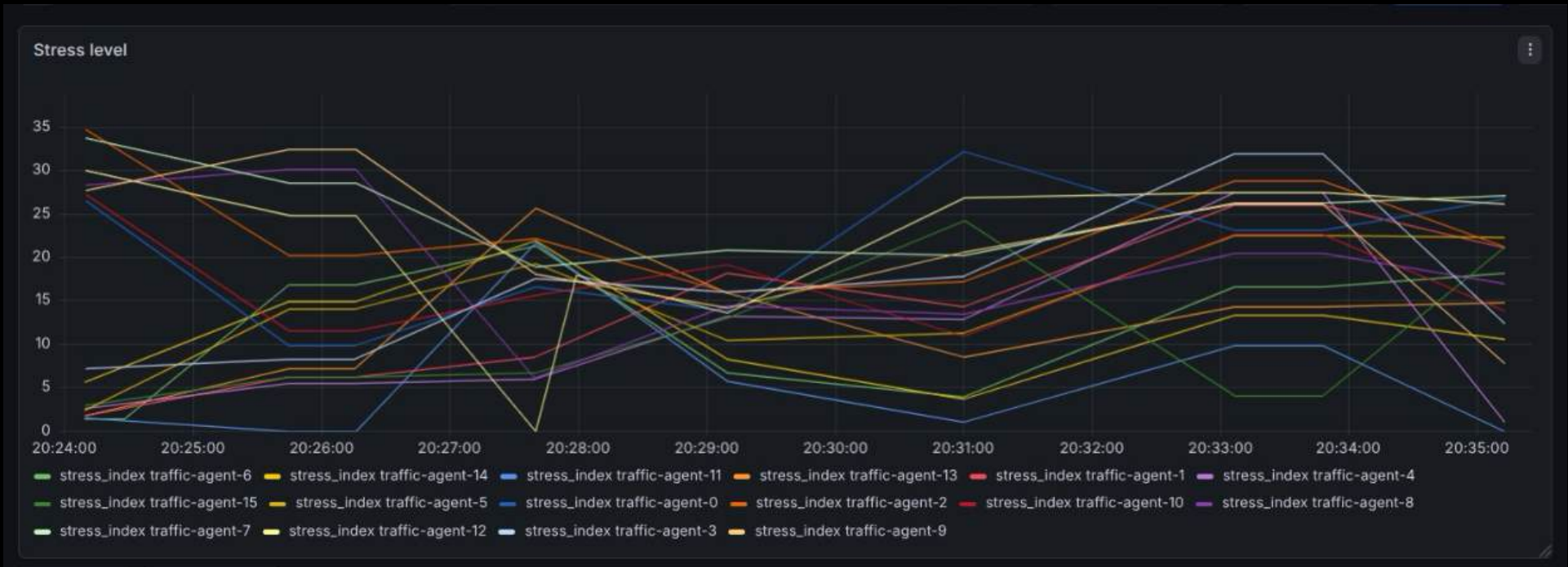
Risultati

—k 4



Risultati Scalabilità

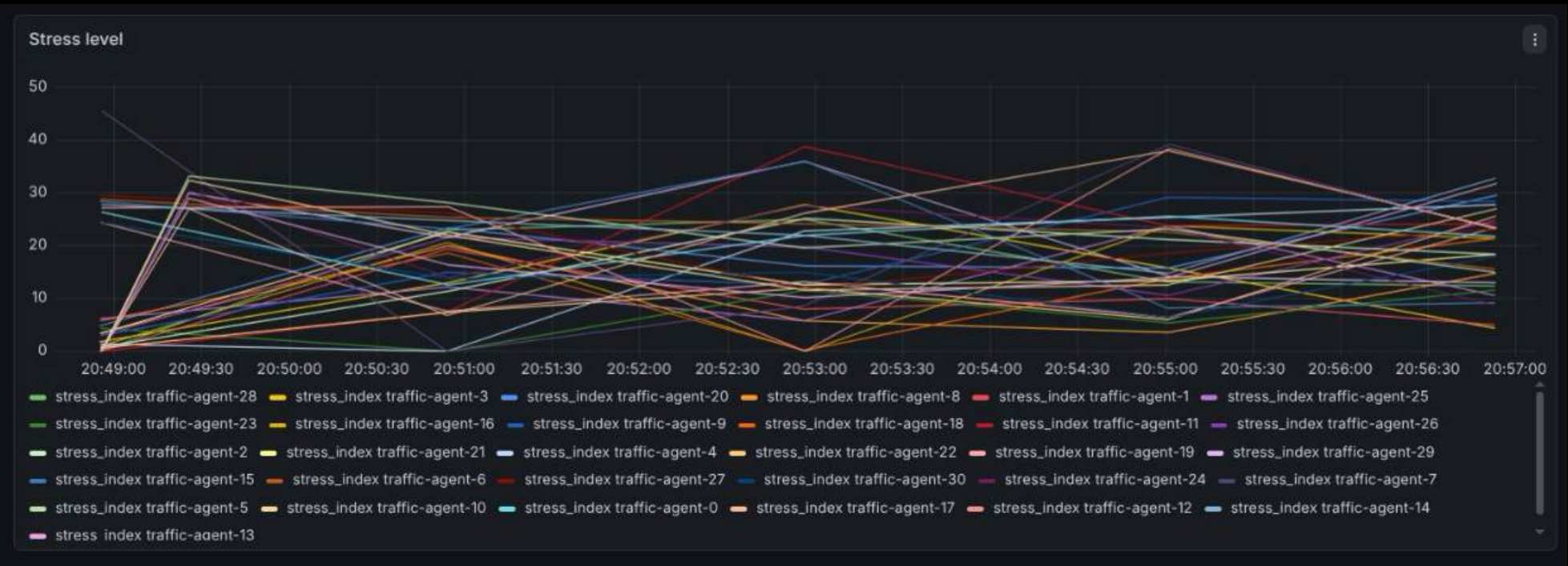
—k 16



—k 8



—k 32



Valutazioni e stime economiche

Software e backend

Agenti	K8s	Back.	Stor.	Net.	SUMO	LLM	Totale
10	40 €	10 €	10 €	3 €	10 €	80 €	~ 158 €
20	80 €	15 €	15 €	5 €	20 €	250 €	~ 395 €
50	180 €	25 €	25 €	10 €	40 €	1000 €	~ 1300 €
100	350 €	40 €	40 €	20 €	80 €	4000 €	~ 4575 €



Valutazioni e stime economiche

LLM token



Valutazioni e stime economiche

Hardware

- Telecamera AI Outdoor (IP66): 250 €
- Edge Device AI (NVIDIA Jetson / mini PC): 450 €
- Installazione hardware (montaggio e cablaggio): 300 €
- Alimentazione e rete (switch PoE / LTE): 150 €

Costo Acquisizione per Singolo Incrocio: 1.150 €

A questo si aggiunge un **Server centrale GPU** (costo fisso condiviso) di 4.000 €.



Formule di Stima Economica

$$1 \quad \text{Costo Acquisizione}(n) = n \times 1.150 \text{ €} + 4.000 \text{ €}$$

$$2 \quad \text{Costo Mensile Mantenimento}(n) = n \times 30 \text{ €} + 20 \text{ €}$$



Valutazioni e stime economiche

Mantenimento

Incroci (n)	Costo Mensile	Formula
10	320 €/mese	$10 \times 30 + 20$
20	620 €/mese	$20 \times 30 + 20$
50	1.520 €/mese	$50 \times 30 + 20$
100	3.020 €/mese	$100 \times 30 + 20$



Valutazioni e stime economiche

Costo downtime

Voce di impatto	Costo stimato
Ritardi e tempi di percorrenza aggiuntivi	1.500 €/ora
Congestione e inefficienza della rete urbana	2.000 €/ora
Perdita monitoraggio e controllo adattivo	500 €/ora
Totale stimato	4.000 €/ora



Valutazioni e stime economiche

Totali stimati

Scenario	Incroci	Agenti	Costo CV	OPEX AI	Mant. HW	Totale 1° Anno
Progetto Pilota	20	7	27.000 €	1.800 €	7.440 €	36.240 €
Area Urbana Centrale	50	17	61.500 €	15.600 €	18.240 €	95.340 €
Copertura Ampia	100	34	119.000 €	31.200 €	36.240 €	186.440 €
Copertura Estesa	150	50	176.500 €	46.800 €	54.240 €	277.540 €



Grazie per l'attenzione